Empirical Evaluation of Hybrid Information Retrieval for Local Developer Knowledge Repositories

Haris Kamal Rana
Department of Computer Science
University of Lahore, Pakistan

Developer knowledge repositories—comprising code, documentation, personal notes, and version-control history—present unique information retrieval challenges due to their heterogeneous structure and the mixture of conceptual and exact-terminology queries that developers issue. We present Nexus, a local-first developer knowledge system, and conduct a systematic empirical evaluation of semantic, lexical, hybrid (via Reciprocal Rank Fusion), and graph-augmented retrieval strategies over a curated corpus of 109 technical documents (1,857 chunks) and 218 validated queries spanning six categories. Our key findings are: (1) hybrid retrieval significantly outperforms both semantic-only and lexical-only baselines on NDCG@5 (0.498 vs. 0.418/0.415, $p < 0.001$); (2) the RRF fusion constant K is remarkably stable across $K \in [5, 200]$; (3) no single chunk configuration dominates all metrics, revealing a precision–recall trade-off; and (4) wikilink-based graph augmentation degrades NDCG@5 under sparse-link conditions (0.467 vs. 0.498, $p < 0.001$)—a negative result we analyze honestly. We further introduce an annotation-ready human-evaluation protocol and document all threats to validity. All experiments are fully offline, deterministic, and reproducible from committed artifacts.

***Index Terms*—information retrieval, hybrid search, reciprocal rank fusion, developer knowledge, empirical evaluation, retrieval-augmented generation**

I. Introduction

Software developers maintain knowledge across multiple fragmented sources: code repositories, API documentation, personal notes, debugging logs, and version-control history. Retrieving relevant information from these sources is a core bottleneck in developer productivity. Unlike general-purpose document retrieval, developer queries often mix *exact technical tokens* (e.g., function names, error messages, configuration keys) with *conceptual information needs* (e.g., "how does error handling work?"). This heterogeneity demands retrieval systems that can handle both lexical precision and semantic understanding.

Modern retrieval-augmented generation (RAG) pipelines [1] rely on high-quality retrieval as their foundation. Yet empirical evaluations of retrieval quality in developer-oriented settings remain underexplored. Most RAG evaluations focus on open-domain question answering over Wikipedia-scale corpora, rather than the smaller, more structured knowledge repositories that individual developers maintain.

This paper presents Nexus, a local-first developer knowledge system built on Obsidian [2], and conducts a systematic empirical evaluation of four retrieval strategies:

1) **Semantic retrieval** using dense embeddings (MiniLM-L6-v2 [3]).
2) **Lexical retrieval** using exact-term matching ranked by term frequency.
3) **Hybrid retrieval** fusing semantic and lexical rankings via Reciprocal Rank Fusion (RRF) [4].
4) **Graph-augmented hybrid retrieval** extending hybrid results through wikilink expansion.

We investigate four research questions:

- **RQ1:** How does hybrid retrieval compare with semantic-only and lexical-only retrieval?
- **RQ2:** How sensitive is retrieval quality to the RRF fusion constant K?
- **RQ3:** How does chunk configuration affect retrieval quality?
- **RQ4:** Does wikilink-based graph augmentation improve retrieval quality in the evaluated developer knowledge corpus?

Our contributions are: (1) empirical evidence that hybrid retrieval outperforms individual strategies on a technical documentation corpus with rigorous statistical testing; (2) a negative result on graph-augmented retrieval under sparse-link conditions, analyzed honestly; (3) a systematic characterization of parameter sensitivity (K and chunk size); and (4) a reproducible evaluation framework with an annotation-ready human-evaluation protocol.

A. Scope and Positioning

This paper does *not* claim that Nexus solves knowledge management in general. The evaluation is conducted over a specific corpus of 109 permissively licensed technical documents from three sources (CPython, FastAPI, SQLite documentation), using 218 synthetic queries with machine-generated ground truth. The findings apply to this experimental configuration and should be interpreted within these boundaries.

The remainder of this paper is organized as follows: Section II reviews relevant concepts; Section III describes the Nexus architecture; Section IV details the retrieval methods; Section V describes the experimental methodology; Section VI presents the results; Section VII discusses findings; Section VIII addresses threats to validity; Section IX documents reproducibility; and Section X concludes.

II. Background

A. Semantic Retrieval

Semantic retrieval represents documents and queries as dense vectors in a shared embedding space, retrieving results by cosine similarity. Pre-trained language models such as BERT [5] and its distilled variants [3] produce embeddings that capture contextual meaning, enabling retrieval based on conceptual similarity rather than exact lexical overlap. Sentence-transformer models [6] further refine embeddings for sentence-level similarity tasks. Semantic retrieval excels at paraphrase-tolerant queries but may miss exact technical identifiers.

B. Lexical Retrieval

Lexical retrieval matches documents based on shared terms. The classical TF-IDF [7] and BM25 [8], [9] frameworks rank documents by term frequency weighted by inverse document frequency. In developer-oriented corpora, lexical retrieval is particularly effective for queries containing exact function names, error messages, or configuration tokens—terms that carry precise technical meaning and may not be well-represented in semantic embedding spaces.

C. Hybrid Retrieval

Hybrid retrieval combines semantic and lexical signals to leverage their complementary strengths. Semantic retrieval handles conceptual queries, while lexical retrieval provides precision for exact-match queries. The challenge lies in fusing ranked lists from heterogeneous similarity spaces without requiring score calibration.

D. Reciprocal Rank Fusion

Reciprocal Rank Fusion (RRF) [4] addresses the fusion problem by operating on document ranks rather than scores. For a document d appearing in n ranked lists, the RRF score is:

$$\text{RRF}(d) = \sum_{i=1}^{n} \frac{1}{K + \text{rank}_i(d)} \tag{1}$$

where $\text{rank}_i(d)$ is the rank of d in the i-th list, and K is a constant (typically 60) that controls the relative influence of rank position. RRF is attractive because it requires no score normalization and is robust to differences in similarity scales between retrieval methods [10].

E. Graph-Augmented Retrieval

Graph-augmented retrieval extends traditional search by leveraging document relationships encoded in a knowledge graph. In the context of developer documentation, wikilinks (hyperlinks between Markdown documents) provide a lightweight graph structure. Graph-augmented systems retrieve initial results, then expand the candidate set by following graph edges to related documents [11]. The effectiveness depends critically on graph density and the semantic relevance of linked documents.

F. Retrieval Evaluation

Standard IR evaluation uses metrics computed against a ground-truth relevance judgment set (qrels). Recall@K measures the fraction of relevant documents in the top-K results. Mean Reciprocal Rank (MRR) measures the inverse rank of the first relevant result. Normalized Discounted Cumulative Gain at K (NDCG@K) [12] accounts for the position of relevant documents in the ranking, assigning higher weight to top-ranked results. Statistical comparisons between systems typically use paired non-parametric tests such as the Wilcoxon signed-rank test [13].

III. System Architecture

Nexus is a local-first developer knowledge system built on Obsidian [2]. Figure 1 shows the high-level architecture. The system comprises four main components: knowledge ingestion, storage, retrieval, and grounded generation.

A. Knowledge Ingestion

Documents enter the system through two pipelines: (1) *Vault watcher*: monitors an Obsidian vault for changes and processes new or modified Markdown files; (2) *Git capture*: automatically generates notes from Git commit diffs via a post-commit hook. Both pipelines feed into the same chunking and indexing pipeline.

B. Chunking

Markdown documents are split at heading boundaries, preserving document structure. Each chunk carries metadata including its source file, heading path, and extracted wikilinks. The default configuration uses 1,200 characters per chunk with 150-character overlap.

C. Embeddings and Storage

Chunks are embedded using MiniLM-L6-v2 (384-dimensional, ONNX runtime) and stored in Chroma [14], a persistent vector database with concurrency-safe writes. The storage layer implements file-level locking and bounded lock timeouts to support safe concurrent access.

D. Retrieval

The retrieval module (`nexus.knowledge.retrieval`) implements three strategies:

- **Semantic**: Chroma vector search using cosine similarity.
- **Lexical**: Chroma `where_document` filtering with case-insensitive regex matching on query terms, ranked by term frequency.
- **Hybrid**: RRF fusion of semantic and lexical ranked lists.

E. Grounded Generation

The agent module composes retrieved context into grounded prompts for a configurable LLM backend (local Ollama or cloud providers via OpenAI-compatible API). This component is not evaluated in this paper; the focus is on retrieval quality.

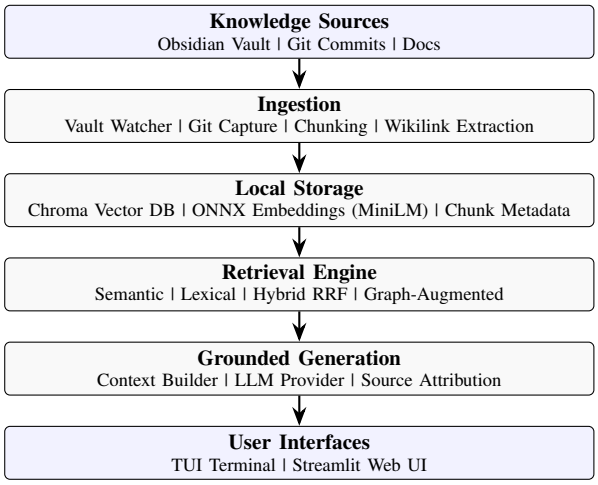

Fig. 1. High-level architecture of Nexus. The retrieval evaluation in this paper focuses on the Semantic/Lexical/RRF pipeline within the retrieval engine.

F. Evaluation Laboratory

The evaluation module (`nexus.knowledge.evaluation`) provides a complete experimental pipeline: corpus construction, query generation, metric computation (Recall@K, MRR, NDCG@5), statistical comparison (paired Wilcoxon signed-rank tests), figure generation, and report generation. All experiments are deterministic and reproducible from committed artifacts.

IV. Retrieval Methods

A. Semantic Retrieval

The semantic retrieval strategy embeds the query using MiniLM-L6-v2 and performs cosine similarity search against the Chroma vector store. Given a query q, the embedding model produces a vector $\mathbf{e}_q \in \mathbb{R}^{384}$. Chroma returns the K chunks with the highest cosine similarity:

$$\text{sim}(q, c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\| \|\mathbf{e}_c\|} \tag{2}$$

where c is a chunk and $\mathbf{e}_c$ is its pre-computed embedding.

B. Lexical Retrieval

The lexical strategy performs exact-term matching using Chroma's `where_document` filtering. The query is tokenized, normalized (lowercased, punctuation removed), and each term is matched against chunk text using case-insensitive regex. Matching chunks are ranked by the count of matched terms (term frequency). This strategy is approximately 5× faster than semantic retrieval (111 ms vs. 566 ms per query).

C. Hybrid Retrieval (RRF)

The hybrid strategy combines semantic and lexical retrieval using Reciprocal Rank Fusion [4]. Each strategy produces a ranked list of candidate chunks. RRF computes a combined score using Equation 1 with default $K = 60$. Duplicates are removed by matching on `(source_file, text_prefix)`, preserving the highest-ranked occurrence.

The key advantage of RRF is that it operates on ranks, not raw scores, avoiding the need to calibrate between heterogeneous similarity spaces (cosine similarity vs. term frequency).

D. Graph-Augmented Retrieval

Graph augmentation extends hybrid retrieval by resolving wikilinks from the top-retrieved chunks. For each top-retrieved chunk, extracted wikilinks are resolved to their target documents, and chunks from those documents are appended to the candidate set. This strategy is *experiment-only* and not part of production retrieval. It investigates whether document relationship structure provides additional retrieval signal.

The expansion is controlled by a parameter expand_k that limits the number of chunks fetched from each linked document. Graph augmentation is expected to help when linked documents are semantically relevant to the query; it may hurt when links are navigational rather than topical.

E. Deduplication

All retrieval strategies deduplicate results by source document. When multiple chunks from the same document appear in the result list, only the highest-ranked chunk is retained for document-level metric computation. This ensures that Recall@K and NDCG@K measure the ability to surface *relevant documents*, not just relevant passages.

V. Experimental Methodology

A. Dataset

The `nexus-dev-ir-v1` evaluation corpus comprises 109 Markdown documents sourced from three permissively licensed technical documentation projects:

- **CPython 3.12** (HowTo + Library docs): 38 documents, PSF-2.0 license
- **FastAPI 0.115** (Tutorial + Advanced + Dependencies): 65 documents, MIT license
- **SQLite** documentation: 6 documents, public domain

Documents are chunked at heading boundaries with a default configuration of 1,200 characters per chunk and 150-character overlap, producing 1,857 chunks. The corpus contains 93 wikilinks. All source documents are committed; the dataset can be rebuilt from pinned upstream versions using the acquisition scripts.

B. Query Construction

The query set consists of 218 queries generated by deterministic templates over corpus structure (headings, code identifiers, titles). Six query categories reflect different information-seeking behaviors:

C. Ground Truth

Relevance labels are generated by deterministic term-overlap validation: a document is labeled as relevant when at least 50% of the query's content terms appear in it, with a minimum of 2 matching terms. Documents matching more than 25% of the full corpus are excluded to avoid trivially relevant results.

TABLE I
QUERY CATEGORIES AND DISTRIBUTION

Category	Count	Template pattern
Conceptual	38	"What is the purpose of X?"
Technical	33	"How does X work?"
Exact	29	"What is X?" (identifier)
Implementation	55	"How do I call X?"
Architecture	14	"What is the structure of X?"
Troubleshooting	49	"What causes X to fail?"
Total	**218**	

TABLE II
RETRIEVAL CONFIGURATIONS

Configuration	Description
Semantic	Dense embedding search (MiniLM-L6-v2)
Lexical	Exact-term matching ranked by term frequency
Hybrid	RRF fusion (K=60) of semantic + lexical
Graph-Hybrid	Hybrid + wikilink expansion (expand_k=3)

This ground truth method is **explicitly lexically biased**: it favors documents that share the query's exact vocabulary. This biases results toward lexical retrieval and should be considered when interpreting the relative performance of different strategies. The queries are synthetic, not drawn from real user information needs. These limitations are discussed further in Section VIII.

D. Retrieval Configurations

We evaluate four main configurations:

Additionally, we conduct three ablation experiments: (1) RRF K sweep over $\{1, 5, 10, 30, 60, 100, 200\}$; (2) chunk-size sweep over $\{(1000, 125), (1200, 150), (2400, 300)\}$ (size, overlap); (3) graph expansion depth sweep over $\{1, 3, 5\}$.

E. Evaluation Metrics

All metrics operate at the *document level*: a retrieved chunk counts as a hit when its source document appears in the relevant set. We report:

- **Recall@K** ($K \in \{1, 3, 5\}$): fraction of relevant documents in the top-K unique source documents.
- **MRR**: reciprocal rank of the first relevant document.
- **NDCG@5**: Normalized Discounted Cumulative Gain with binary gains, truncated at 5 [12].

Aggregates report mean, median, standard deviation, and 95% bootstrap confidence intervals (2,000 resamples, seeded). Latency is measured as wall-clock time per query.

F. Statistical Testing

Paired comparisons use the **Wilcoxon signed-rank test** [13] on per-query metric values (two-sided, zero differences dropped, normal approximation for $n > 30$). Results are reported as descriptive evidence; we do not claim population-level significance given the synthetic nature of the query set.

TABLE III
BASELINE RETRIEVAL COMPARISON. BEST VALUES IN BOLD.

Strategy	R@1	R@3	R@5	MRR	NDCG@5	ms/q
Semantic	.188	.286	.331	.669	.418	566
Lexical	.160	.249	.303	.588	.415	111
Hybrid	**.207**	**.309**	**.373**	**.735**	**.498**	641

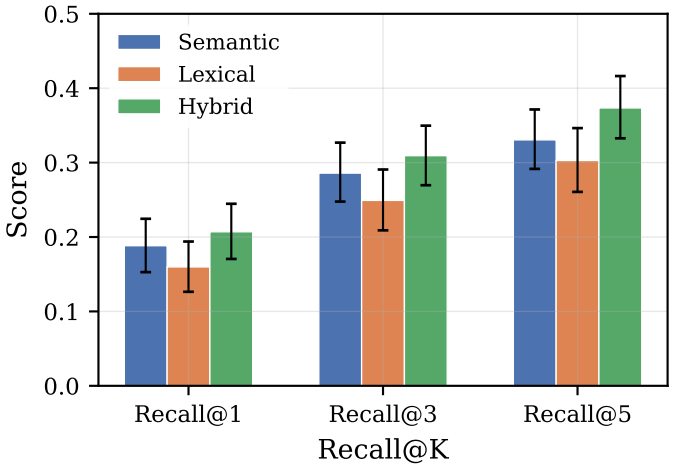

Fig. 2. Recall@K comparison across strategies with 95% bootstrap confidence intervals.

G. Experimental Environment

All experiments run on CPU (no GPU) in a fully offline, deterministic configuration with a fixed random seed (2026). The graph ablation experiment uses deterministic hash-based embeddings for controlled comparison across expansion depths; all other experiments use ONNX MiniLM embeddings.

VI. RESULTS

A. Baseline Retrieval Comparison

Table III presents the baseline comparison across all metrics. Hybrid retrieval achieves the best performance on every metric. Lexical retrieval shows the lowest latency (111 ms) but the weakest recall. Semantic retrieval achieves slightly higher recall than lexical but at significantly higher latency (566 ms).

Figure 2 shows the Recall@K comparison with 95% bootstrap confidence intervals. The hybrid strategy's advantage is consistent across $K = 1, 3, 5$.

1) Statistical Comparisons: Table IV reports paired Wilcoxon signed-rank tests on per-query NDCG@5. Hybrid significantly outperforms both semantic ($p < 0.001$) and lexical ($p < 0.001$). Semantic and lexical are not significantly different from each other ($p = 0.819$), suggesting they capture complementary but roughly equal signal strength in this configuration.

B. RRF Fusion Constant Sensitivity

Figure 3 shows retrieval quality across $K \in \{1, 5, 10, 30, 60, 100, 200\}$. Performance is remarkably stable for $K \geq 5$. The NDCG@5 ranges from 0.484 (K=1)

TABLE IV
PAIRED WILCOXON SIGNED-RANK TESTS (NDCG@5).

Comparison	W	p-value	n
Hybrid > Semantic	739	< 0.001	109
Hybrid > Lexical	3,411	< 0.001	148
Semantic $\approx$ Lexical	6,332	$= 0.819$	157

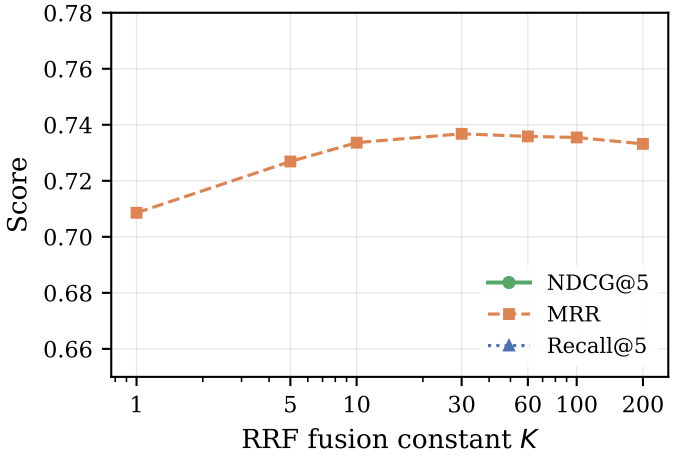

Fig. 3. RRF fusion constant K sensitivity. Performance is stable for $K \in [5, 200]$.

TABLE V
CHUNK-SIZE ABLATION. BEST VALUES PER METRIC IN BOLD.

Size	Ov.	Chunks	R@1	R@5	MRR	NDCG@5
1,000	125	2,213	**.214**	.358	**.742**	.488
1,200	150	1,857	.207	.373	.735	.498
2,400	300	961	.198	**.377**	.737	**.503**

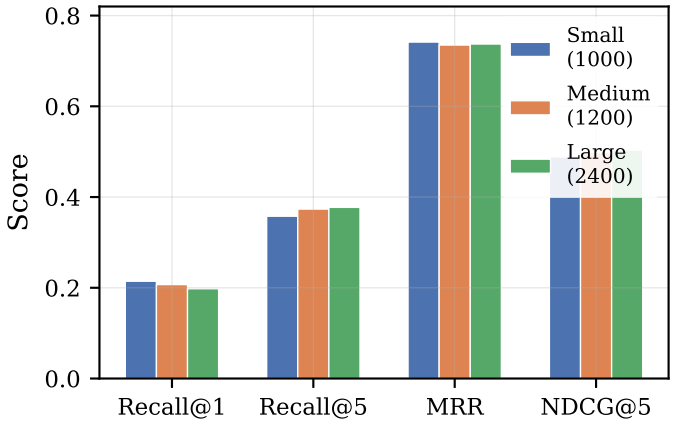

Fig. 4. Chunk-size comparison across metrics. No single configuration dominates all metrics.

to 0.501 (K=60), a variation of less than 0.02. Recall@5 is constant at $\approx$ 0.37–0.38 for $K \geq 5$. Only $K = 1$ shows minor degradation, consistent with the theoretical expectation that very small K values over-weight the first-ranked list.

C. Chunk Configuration

Figure 4 and Table V present the chunk-size ablation. No single configuration dominates all metrics:

- **Small (1,000/125):** Best Recall@1 (0.214) and MRR (0.742). Smaller chunks surface the most relevant document first.
- **Medium (1,200/150):** Balanced performance; production default.
- **Large (2,400/300):** Best Recall@5 (0.377) and NDCG@5 (0.503). Larger chunks contain more context for multi-document relevance.

D. Graph-Augmented Retrieval

Figure 5 and Table VI compare hybrid retrieval with graph-augmented hybrid retrieval.
Graph augmentation slightly improved Recall but **significantly degraded NDCG@5** (0.498 $\rightarrow$ 0.467, Wilcoxon $p < 0.001$, n=90 non-zero differences). This negative result is discussed in Section VII-E.

E. Graph Expansion Ablation

To investigate the conditions under which graph expansion helps or hurts, we conducted a controlled ablation varying the expansion depth (expand_$k \in \{1, 3, 5\}$) using deterministic hash embeddings.

Figure 6 shows that all graph expansion depths degrade NDCG@5 relative to the baseline: 0.256 (baseline) vs. 0.240 (k=1), 0.251 (k=3), 0.252 (k=5). Even minimal expansion (k=1) displaces higher-ranked base results. Larger expansion partially recovers NDCG@5 but never exceeds the baseline. The controlled ablation confirms that the degradation observed in Section VI-D is not an artifact of a single configuration.

Note that the graph ablation experiment uses hash-based embeddings (not MiniLM), so absolute metric values differ from the baseline experiment. The *relative* comparison between baseline and graph-expanded configurations within this experiment remains valid.

F. NDCG Implementation Validation

During development, the evaluation pipeline underwent validation and correction. The original NDCG implementation incorrectly counted duplicate source documents multiple times when computing DCG, while the ideal ranking used unique documents. This could produce NDCG values greater than 1.0. The implementation was corrected to deduplicate source documents before calculating NDCG. All experiments reported in this paper use the corrected implementation. This process is documented as an example of metric validation in the experimental pipeline.

VII. DISCUSSION

A. Why Hybrid Retrieval Works

Hybrid retrieval succeeds because semantic and lexical search capture fundamentally different aspects of relevance in technical corpora. **Semantic retrieval** captures conceptual similarity: a query about "error handling" matches documentation about

TABLE VI
HYBRID VS. GRAPH-AUGMENTED HYBRID.

Strategy	R@1	R@3	R@5	MRR	NDCG@5
Hybrid	.207	.313	.373	.735	**.498**
Graph-Hybrid	.210	.321	.379	.735	.467

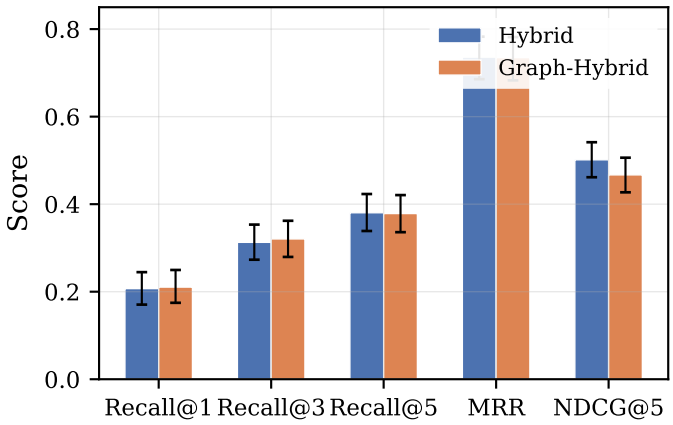

Fig. 5. Hybrid vs. graph-augmented hybrid. Graph augmentation degrades NDCG@5 despite marginally improving recall.

exception handling, even when the exact word "error" is absent. **Lexical retrieval** captures exact terminology: a query for "jsonable_encoder" finds the specific function documentation even if the embedding model does not prioritize it.

The observation that semantic and lexical achieve similar NDCG@5 (0.418 vs. 0.415, $p = 0.82$) but through different mechanisms explains why their combination is effective. RRF fusion is well-suited to this setting because it operates on ranks, avoiding the need to calibrate between cosine similarity scores and term frequencies. The resulting hybrid is never worse than either component on NDCG@5 and achieves the best recall.

B. Why Lexical Retrieval Is Strong on Technical Corpora

The evaluated corpus is dominated by technical documentation where exact identifiers (function names, class names, error codes) are central to queries. Lexical retrieval's strength in matching these exact tokens explains its competitive performance despite its conceptual simplicity. The 5× latency advantage (111 ms vs. 566 ms) further supports lexical retrieval as a viable baseline for latency-sensitive applications.

C. RRF Parameter Robustness

The remarkable stability of RRF across $K \in [5, 200]$ (Figure 3) has practical implications. In production systems, the choice of K is often treated as a sensitive hyperparameter requiring tuning. Our results suggest that for this class of problems, any K in a wide range yields similar performance. The production default of $K = 60$ is reasonable but not uniquely optimal.

D. Chunk-Size Trade-offs

The chunk-size ablation reveals a fundamental tension in RAG systems. Small chunks (1,000 characters) improve first-result precision because each chunk is more focused, making it more likely that the most relevant content is surfaced at rank 1. Large chunks (2,400 characters) improve recall and NDCG because each chunk contains more context, increasing the probability that a chunk will be judged relevant. The "right" size depends on the application: code search may favor smaller chunks, while documentation Q&A may favor larger ones. The medium configuration (1,200) represents a reasonable default that balances both objectives.

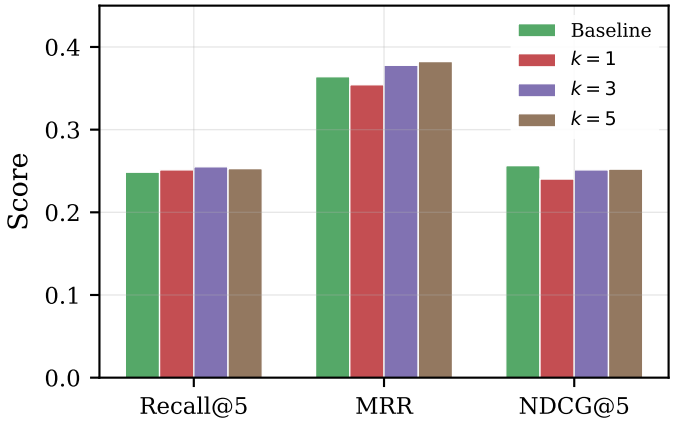

Fig. 6. Graph expansion ablation. All expansion depths degrade NDCG@5 relative to the hybrid baseline.

E. Why Graph Augmentation Failed

The negative graph result (Sections VI-D–VI-E) is one of the most informative findings in this evaluation. We identify four contributing factors:

1) **Sparse connectivity.** The corpus contains 93 wikilinks across 109 documents (approximately 0.85 links per document). Most documents have 0–2 incoming or outgoing links, providing weak graph structure for expansion.
2) **Navigational vs. semantic links.** Wikilinks in developer documentation are primarily navigational ("see also"), not semantic ("is relevant to the query"). The graph encodes documentation structure, not knowledge relationships.
3) **Query–document mismatch.** Graph expansion retrieves documents relevant to the *source chunk's topic*, not the *query's information need*. A query about "asyncio.run" might expand to a linked document about "threading"—related topics, but not what the user asked about.
4) **Candidate displacement.** The expanded candidates displace higher-ranked semantic/keyword matches from the top-K result set, degrading ranking quality (NDCG) even as they marginally improve recall.

This negative result is academically valuable because it demonstrates that the experiments were designed to measure actual performance rather than to manufacture positive results. It also suggests that graph-augmented retrieval may require denser knowledge graphs with more semantically meaningful

edges (e.g., API cross-references, dependency chains) to be beneficial.

F. Latency Trade-offs

Semantic retrieval (566 ms) and hybrid retrieval (641 ms) incur meaningful latency on CPU. For interactive applications, this may require caching, approximate nearest neighbor search, or GPU acceleration. Lexical retrieval (111 ms) is fast enough for real-time interaction without optimization. The graph-augmented strategy adds negligible overhead to the retrieval itself but does not reduce latency in the overall pipeline.

VIII. Threats to Validity

A. Internal Validity

1) **Lexical ground-truth bias.** The term-overlap validation method inherently favors documents that share the query's exact vocabulary. This biases results toward lexical retrieval and may overstate the performance gap between strategies. Human relevance judgments would provide a more balanced evaluation (Section IX).
2) **Synthetic queries.** All 218 queries are generated by deterministic templates, not drawn from real user information needs. Template-generated queries may not reflect the natural language variation or specificity of actual developer queries.
3) **Single embedding model.** All experiments use MiniLM-L6-v2. Results may differ with larger models (e.g., all-mpnet-base-v2), domain-specific models, or models fine-tuned on technical text.
4) **Document-level relevance judgments.** A "relevant" document is one whose chunks should surface, not a passage-level judgment. This is coarser than standard passage-level IR evaluation and may miss fine-grained relevance distinctions.

B. External Validity

1) **Limited corpus size.** The evaluation corpus contains 109 documents from 3 sources. Performance may differ substantially on larger or more diverse corpora.
2) **Domain bias.** All sources are technical documentation (Python, FastAPI, SQLite). Results may not generalize to other domains such as academic papers, code comments, or internal company documentation.
3) **Limited graph structure.** The wikilink graph is sparse (93 links, ≈ 0.85 links/document). Denser knowledge graphs with more semantically meaningful edges might benefit from graph augmentation.
4) **External generalization.** These findings describe the behavior of specific retrieval implementations on a specific corpus. They should not be taken as universal claims about the relative merits of semantic, lexical, and hybrid retrieval.

C. Construct Validity

1) **Binary relevance.** Machine-generated labels are binary (relevant/irrelevant). Graded human judgments may reveal different patterns in retrieval quality.
2) **NDCG@5 resolution.** With small relevant sets per query (typically 1–9 documents), NDCG@5 can be noisy. Multiple queries with 1–2 relevant documents reduce the metric's resolution.
3) **Document-level qrels.** Standard IR evaluation typically uses passage-level or document-level relevance. Our document-level approach is consistent within the evaluation but differs from some standard benchmarks.

D. Reliability

1) **CPU/platform-dependent latency.** Latency measurements are hardware-dependent and reported for completeness only. They do not claim cross-platform reproducibility.
2) **Single annotator.** The human evaluation framework (Section IX) is designed for a single annotator (the developer). Inter-annotator agreement is not measured.
3) **Deterministic but not generalizable.** All experiments use a fixed random seed (2026). Different seeds might yield slightly different rankings for edge cases, though aggregate metrics should be stable.

IX. Reproducibility

All experiments are fully reproducible from committed artifacts. The repository at https://github.com/harisrana-dev/nexus contains the complete source code, dataset, and result files.

A. Dataset

The `nexus-dev-ir-v1` dataset (corpus and queries) is committed at `research/datasets/nexus-dev-ir-v1/`. The corpus is built from pinned upstream versions (CPython v3.12.7, FastAPI 0.115.6, SQLite docs). Raw downloads are reproducible via the acquisition script (network required once).

B. Experiment Commands

The evaluation laboratory is invoked via `python -m nexus.knowledge.evaluation`. The following commands reproduce the results in this paper:

```
# Run the full experiment suite (ONNX embeddings,
python -m nexus.knowledge.evaluation run-all

# Individual experiments
python -m nexus.knowledge.evaluation baseline
python -m nexus.knowledge.evaluation rrf_sweep
python -m nexus.knowledge.evaluation chunking_swee
python -m nexus.knowledge.evaluation graph_rag

# Graph ablation (hash embeddings, controlled comp
python -m nexus.knowledge.evaluation graph_ablatio

# Human evaluation workflow
```

```
python -m nexus.knowledge.evaluation generate-human
# ... annotate candidates ...
python -m nexus.knowledge.evaluation evaluate-human

# Regenerate figures and reports
python -m nexus.knowledge.evaluation plots
python -m nexus.knowledge.evaluation report
```

C. Result Artifacts

All result files are committed at `research/results/`:

- `baseline.json` – semantic vs. lexical vs. hybrid
- `rrf_sweep.json` – RRF K ablation
- `chunking_sweep.json` – chunk-size ablation
- `graph_rag.json` – hybrid vs. graph-hybrid
- `graph_ablation.json` – graph expansion depth ablation

D. Deterministic Components

The following components are fully deterministic:

- Corpus construction (pinned upstream versions)
- Query generation (seeded random templates)
- Metric computation (seeded bootstrap CIs)
- Figure and report generation

Latency measurements are inherently hardware-dependent and should be interpreted as relative comparisons within the same experimental environment.

E. Human Evaluation Protocol

The annotation-ready human evaluation framework is implemented and documented. The workflow is:

1) **Candidate generation:** For each query, the top-8 retrieved documents across strategies are collected.
2) **Annotation:** The developer reviews candidates and assigns graded relevance: 0 (irrelevant), 1 (partially relevant), 2 (highly relevant).
3) **Evaluation:** Once annotations exist, standard metrics are computed against human judgments.

The candidate file has been generated (`research/datasets/nexus-human-eval-v1/candidates.json`) with 94 representative queries across all six categories. The annotation template is provided. No human annotations have been completed yet; the results reported in this paper use only machine-generated ground truth. This limitation is acknowledged throughout the paper.

X. Conclusion

This paper presented an empirical evaluation of semantic, lexical, hybrid (RRF), and graph-augmented retrieval strategies for local developer knowledge repositories. Over a curated corpus of 109 technical documents and 218 validated queries, we found:

1) **Hybrid retrieval is the strongest overall strategy,** achieving NDCG@5 of 0.498 versus 0.418 (semantic) and 0.415 (lexical), with statistically significant improvements ($p < 0.001$) over both baselines.
2) **RRF fusion is robust to parameter choice,** with stable performance across $K \in [5, 200]$. The production default $K = 60$ is reasonable but not uniquely optimal.
3) **Chunk configuration involves a precision–recall trade-off,** with no single configuration dominating all metrics. Small chunks favor first-result precision; large chunks favor recall and NDCG.
4) **Graph augmentation degrades ranking quality** under the evaluated sparse-link conditions (NDCG@5: 0.467 vs. 0.498). The controlled ablation confirms this is not an artifact of a single configuration.

The negative graph result is a contribution in itself: it demonstrates honest reporting and identifies the conditions under which graph-augmented retrieval is unlikely to help—specifically, sparse navigational link graphs that do not encode semantic relevance.

A. Significance

This study provides empirical evidence for retrieval strategy selection in developer knowledge systems. The results are specific to the evaluated corpus, query set, and experimental configuration, but they offer grounded guidance: hybrid retrieval is a strong default, RRF is parameter-insensitive, and graph augmentation requires denser, semantically meaningful knowledge graphs.

B. Future Work

Several directions extend this work:

- **Human relevance judgments.** The annotation-ready framework is in place; completing the annotation would provide ground truth without lexical bias.
- **Denser knowledge graphs.** Replacing wikilinks with API cross-references, dependency chains, or LLM-generated relevance links could make graph augmentation effective.
- **Stronger embedding models.** Evaluating larger models (all-mpnet-base-v2, E5, Cohere embed) and domain-specific fine-tuning could reveal whether semantic retrieval's gap relative to hybrid narrows with better embeddings.
- **Cross-encoder reranking.** Applying a cross-encoder reranker after initial retrieval could improve precision without the noise introduced by graph expansion.
- **Larger and more diverse corpora.** Extending the evaluation to internal company documentation, code repositories, or mixed-domain corpora would test generalizability.
- **Real user queries.** Replacing synthetic templates with actual developer queries would improve ecological validity.

All of these directions build on the reproducible evaluation framework established in this paper.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[2] O. contributors, “Obsidian: A second brain, forever private, local,” https://obsidian.md, 2024.
[3] W. Wang, R. Liao, and F. Wei, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 5776–5788.
[4] G. V. Cormack, C. L. A. Clarke, and S. R. Butt, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, 2019, pp. 4171–4186.
[6] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
[7] K. Sparck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.
[8] S. E. Robertson, S. Walker, K. S. Jones, M. M. Hancock-Beaulieu, and M. Gatford, “Okapi at TREC-3,” *NIST Special Publication*, vol. 500-207, pp. 109–126, 1994.
[9] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
[10] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
[11] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” *arXiv preprint arXiv:2404.16130*, 2024.
[12] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.
[13] F. Wilcoxon, *Individual Comparisons by Ranking Methods*. Johns Hopkins Press, 1945.
[14] C. contributors, “Chroma: The open-source AI application embedding database,” https://github.com/chroma-core/chroma, 2024.